Java Concurrency
Complete Interview Question Bank

For 3-5 Years Java Backend Developer Interviews
Every question below, in simple words, with a ready-to-say answer for each.

Total Questions Covered	78
Concept / Theory Questions	68
Coding / Whiteboard Questions	10
Topics	15 Levels + Quick Comparison Round

How this guide is organized

This covers **only** the topics from your Java Concurrency interview roadmap — nothing outside it. Every single question includes two parts:

- **What it means** — a simple-words explanation so you actually understand the concept, not just memorize it.
- **How to answer** — a ready-to-say response you can practice out loud, phrased the way you'd actually speak to an interviewer.

Tip: Read a question, try answering it yourself first in your own words, THEN check the explanation and answer script. That's how you actually retain it instead of just re-reading.

Contents

LEVEL 1 - Core Thread Concepts (8 questions)
LEVEL 2 - Synchronization (7 questions)
LEVEL 3 - wait(), notify(), notifyAll() (3 questions)
LEVEL 4 - Executor Framework (5 questions)
LEVEL 5 - CompletableFuture (5 questions)
LEVEL 6 - Concurrent Collections (4 questions)
LEVEL 7 - Atomic Classes (4 questions)
LEVEL 8 - Locks (ReentrantLock, ReadWriteLock, StampedLock) (3 questions)
LEVEL 9 - Volatile (4 questions)
LEVEL 10 - Java Memory Model (JMM) (4 questions)
LEVEL 11 - Deadlock (4 questions)
LEVEL 12 - ForkJoin Framework (3 questions)
LEVEL 13 - Virtual Threads (Java 21) (5 questions)
LEVEL 14 - Structured Concurrency (3 questions)
LEVEL 15 - ThreadLocal & Scoped Values (3 questions)
QUICK COMPARISON ROUND (3 questions)
Frequently Asked Coding Questions (10 questions)

LEVEL 1 - Core Thread Concepts

The absolute basics. If you fumble here, the interviewer assumes you don't know concurrency at all.

Q1

What is a thread?

What it means:

A thread is the smallest unit of execution inside a program. One process (your running Java application) can have many threads, and all of them share the same memory space but run independently of each other.

How to answer:

"A thread is a lightweight sub-process — the smallest unit of CPU execution. Multiple threads inside one process share the same memory, which lets them do work in parallel."

Q2

Difference between Process and Thread?

What it means:

A process is a fully independent running program with its own separate memory. A thread lives inside a process and shares that process's memory with other threads. Creating a thread is much cheaper than creating a process.

How to answer:

Give it as a quick table in your head: Process = own memory, heavy, isolated. Thread = shared memory, light, fast to create. Say "threads are cheaper because they don't need a new memory space, just a new stack."

Q3

How do you create a thread?

What it means:

Two common ways: (1) extend the Thread class and override run(), or (2) implement the Runnable interface and pass it into a Thread object. In modern code you'd typically hand a Runnable/Callable to an ExecutorService instead of creating raw threads.

How to answer:

Mention both ways briefly, then add: "In real projects I avoid creating raw Thread objects — I submit tasks to an ExecutorService instead, so the pool manages lifecycle for me." This shows practical maturity.

Q4

Why is Runnable preferred over extending Thread?

What it means:

Java allows only single inheritance, so if you extend Thread you can't extend anything else. Implementing Runnable separates "what work to do" from "how it's executed," which is cleaner design and works with thread pools.

How to answer:

"Runnable decouples the task from the thread mechanics — better separation of concerns, and it plugs directly into ExecutorService."

Q5

Difference between Runnable and Callable?

What it means:

Runnable's run() method returns nothing and cannot throw a checked exception. Callable's call() method returns a value and can throw checked exceptions. Callable is used with ExecutorService.submit() to get back a Future.

How to answer:

State the two differences directly, then add an example: "If my task needs to return a computed result, I use Callable with submit(), and get the result via Future.get()."

Q6

What does join() do?

What it means:

join() makes the thread that calls it (usually the main thread) pause and wait until the target thread finishes running.

How to answer:

"I use join() when the main thread depends on a worker thread's result and must wait for it to complete before moving on."

Q7

Difference between sleep() and wait()?

What it means:

sleep() simply pauses the current thread for a fixed time and does NOT release any lock it holds. wait() must be called from inside a synchronized block, and it releases the lock while waiting, resuming only when another thread calls notify()/notifyAll().

How to answer:

The key point interviewers want is the lock release — say it explicitly: "sleep() keeps the lock, wait() releases it. wait() is for inter-thread communication, sleep() is just a pause."

Q8

How does interrupt() work?

What it means:

Calling interrupt() sets an internal "interrupted" flag on the target thread. If that thread is currently blocked in sleep()/wait()/join(), it immediately throws InterruptedException. If it's running normal code, the thread itself has to periodically check Thread.interrupted() and decide to stop.

How to answer:

"It's cooperative, not forceful — interrupt() just requests the thread to stop; the thread's own code has to check for and honor that request."

LEVEL 2 - Synchronization

Almost guaranteed to come up — this is the core mechanism behind thread safety in Java.

Q9

What is synchronization?

What it means:

Synchronization is a mechanism that controls access to a shared resource so that only one thread can read/modify it at a time, which prevents data corruption from simultaneous access.

How to answer:

Define it in one line, then immediately mention the tool: "Java achieves this mainly through the synchronized keyword or explicit Lock objects."

Q10

Why do race conditions happen?

What it means:

A race condition happens when two or more threads read and write shared data at the same time without any coordination, so the final result depends on unpredictable timing — often causing lost updates.

How to answer:

Use the classic example: "Two threads both read counter=5, both add 1, both write 6 — one increment is lost. That's a race condition."

Q11

What is a monitor?

What it means:

A monitor is the built-in lock every Java object automatically has. The synchronized keyword uses this monitor: a thread must acquire an object's monitor before entering a synchronized block on that object.

How to answer:

"Every object in Java carries an implicit monitor lock — synchronized just acquires and releases it for you."

Q12

Difference between object lock and class lock?

What it means:

An object lock (instance-level synchronized method/block) locks on a specific object instance (this) — different instances can run the method concurrently. A class lock (static synchronized) locks on the Class object itself, so only one thread across ALL instances can run it at a time.

How to answer:

"Object lock = per-instance, class lock = shared across every instance because it locks the .class object."

Q13

How does synchronized work internally?

What it means:

At the bytecode level, synchronized blocks are compiled using monitorenter and monitorexit instructions. When a thread hits monitorenter, it tries to acquire the object's monitor; if another thread already holds it, this thread blocks until it's released.

How to answer:

Mentioning monitorenter/monitorexit shows depth: "synchronized isn't just syntax sugar — it compiles down to explicit monitor acquire/release bytecode instructions."

Q14

Why is ReentrantLock called reentrant?

What it means:

A thread that already holds the lock can acquire it again without blocking itself — for example, calling one synchronized method from inside another synchronized method on the same object. Both synchronized and ReentrantLock are reentrant.

How to answer:

Give the simple example: "If method A is synchronized and calls method B which is also synchronized on the same object, the same thread can enter B without deadlocking itself — that's reentrancy."

Q15

Difference between Lock and synchronized?

What it means:

Lock (from java.util.concurrent.locks) gives you tryLock(), timed locking, interruptible locking, and fairness policies — all things synchronized can't do. But with Lock you must manually unlock() in a finally block, whereas synchronized releases automatically.

How to answer:

"synchronized is simpler and auto-releases; Lock gives finer control like tryLock and fairness, but I have to remember to unlock() in a finally block myself."

LEVEL 3 - wait(), notify(), notifyAll()

A favorite topic for testing real understanding, not memorized definitions.

Q16

Why must wait() be inside synchronized?

What it means:

wait() operates on the object's monitor — it needs to release that monitor while the thread waits. To do that, the calling thread must already hold the monitor, i.e. be inside a synchronized block on that object, otherwise Java throws IllegalMonitorStateException.

How to answer:

"wait() needs to release a lock it's holding — so you must be holding that lock first, which means being inside synchronized."

Q17

When should you use notifyAll()?

What it means:

notify() wakes up only one arbitrary waiting thread, while notifyAll() wakes up all waiting threads. Use notifyAll() when multiple threads could be waiting for different conditions, so you don't accidentally leave a thread waiting forever (a missed-signal problem).

How to answer:

"I use notifyAll() whenever more than one thread might be waiting on the same monitor for different reasons — it's safer even if slightly less efficient than notify()."

Q18

What happens if notify() wakes the wrong thread?

What it means:

The woken thread should always re-check its waiting condition (in a while loop, not an if statement) before proceeding. If the condition still isn't true, it goes back to waiting instead of assuming it's now safe to proceed.

How to answer:

"That's exactly why you always wrap wait() in a while loop that rechecks the condition — never assume being woken means the condition is met."

LEVEL 4 - Executor Framework

Very commonly tested for Spring Boot / backend roles specifically.

Q19

Why use ExecutorService?

What it means:

It manages a reusable pool of threads instead of creating a brand-new thread for every task, which avoids the overhead of thread creation/destruction and lets you control how much concurrency your app uses.

How to answer:

"It abstracts away thread lifecycle management — I submit tasks and the pool handles creation, reuse, and scheduling."

Q20

FixedThreadPool vs CachedThreadPool?

What it means:

FixedThreadPool keeps a fixed number of threads alive with an unbounded task queue — good for predictable, steady workloads. CachedThreadPool creates new threads as needed and reuses idle ones, with no upper bound — good for many short-lived, bursty tasks, but risky since it can create unlimited threads.

How to answer:

"FixedThreadPool is safer for production because it caps resource usage; CachedThreadPool can spiral out of control under heavy load since it has no thread limit."

Q21

What happens if the task queue is full?

What it means:

In a ThreadPoolExecutor with a bounded queue, once the queue is full and the pool is already at its max thread count, new tasks are handled by the configured RejectedExecutionHandler — by default this throws a RejectedExecutionException.

How to answer:

"By default it throws RejectedExecutionException, but you can plug in a custom RejectedExecutionHandler — e.g. CallerRunsPolicy to slow down the submitting thread instead of failing outright."

Q22

Difference between submit() and execute()?

What it means:

execute() takes a Runnable and returns nothing — if the task throws an exception, it's lost/printed to the thread's uncaught exception handler. submit() takes a Runnable or Callable and returns a Future, which lets you retrieve a result or catch exceptions via get().

How to answer:

"submit() gives me a Future so I can check for exceptions and get a return value; execute() is fire-and-forget."

Q23

How do you gracefully shut down a thread pool?

What it means:

Call shutdown() so the pool stops accepting new tasks but finishes ones already queued/running, then call awaitTermination() with a timeout to wait for completion. Use shutdownNow() only if you need to forcibly cancel running tasks.

How to answer:

"I call shutdown() first, then awaitTermination() with a sensible timeout, and fall back to shutdownNow() if it doesn't finish in time — that's the standard graceful-shutdown pattern."

LEVEL 5 - CompletableFuture

Extremely common when the role involves microservices calling other services.

Q24

Difference between Future and CompletableFuture?

What it means:

Future.get() blocks the calling thread until the result is ready, and there's no built-in way to chain callbacks or combine multiple futures. CompletableFuture supports non-blocking chaining (thenApply, thenCompose), combining multiple futures (allOf, thenCombine), and explicit exception handling.

How to answer:

"Future is a passive placeholder for a result — I have to block to get it. CompletableFuture is active — I can chain what happens next without blocking."

Q25

Difference between thenApply() and thenCompose()?

What it means:

thenApply() transforms the result using a function that returns a plain value (like map). thenCompose() chains another function that itself returns a CompletableFuture, flattening nested futures instead of ending up with a CompletableFuture> (like flatMap).

How to answer:

"thenApply is like map — transform a value. thenCompose is like flatMap — chain another async call and flatten the result."

Q26

Difference between thenAccept() and thenRun()?

What it means:

thenAccept() takes the previous result as input (via a Consumer) but returns nothing. thenRun() doesn't get access to the previous result at all — it just runs some code after completion.

How to answer:

"thenAccept uses the result but produces no output; thenRun doesn't even look at the result, it just triggers a follow-up action."

Q27

How do you combine multiple API calls?

What it means:

Use CompletableFuture.allOf() to wait for several independent futures to all complete, or thenCombine() to merge the results of exactly two futures into one value once both finish.

How to answer:

"For calling 3 services in parallel, I'd kick off three supplyAsync() calls, then use allOf() to wait for all of them, or thenCombine() when I only need to merge two."

Q28

How do you handle exceptions in CompletableFuture?

What it means:

Use exceptionally() to catch an error and supply a fallback value, or handle() to process both the success result and any exception together in one callback.

How to answer:

"I use exceptionally() for a simple fallback value, or handle() when I need to look at both the result and the exception in the same place."

LEVEL 6 - Concurrent Collections

Interviewers love asking 'why' here, not just 'what'.

Q29

Why isn't HashMap thread-safe?

What it means:

HashMap has no internal locking at all. If multiple threads write to it concurrently — especially during a resize operation — it can corrupt its internal bucket structure, and in older Java versions could even cause an infinite loop.

How to answer:

"HashMap does zero synchronization internally, so concurrent writes can corrupt its internal structure — that's why we never use it across threads without external locking."

Q30

How does ConcurrentHashMap work?

What it means:

Instead of locking the entire map, it locks only small parts of it — modern versions use a mix of CAS operations and synchronized blocks scoped to individual bins/nodes, so different threads can safely update different parts of the map at the same time.

How to answer:

"It uses fine-grained locking at the bucket/node level rather than one big lock, which is why it scales so much better under concurrent access than a synchronized HashMap."

Q31

Why doesn't ConcurrentHashMap lock the whole map?

What it means:

Because locking the whole structure would force every thread to wait for every other thread even when they're touching completely different keys — defeating the purpose of concurrency. Fine-grained locking lets unrelated operations proceed in parallel.

How to answer:

"Locking everything would serialize all access and kill throughput — fine-grained locking is what actually delivers concurrency."

Q32

When should you use CopyOnWriteArrayList?

What it means:

Use it when reads vastly outnumber writes — for example, a list of event listeners. Every write creates a brand-new copy of the underlying array, so writes are expensive, but reads never need any locking at all and are extremely fast.

How to answer:

"I'd reach for it for something like a listener registry — read constantly, modified rarely — because reads are lock-free but writes are costly."

LEVEL 7 - Atomic Classes

Ties directly into CAS and lock-free programming — a favorite deep-dive area.

Q33

Why use AtomicInteger?

What it means:

It provides thread-safe operations like increment, decrement, and compare-and-set on a single variable without needing a lock — internally it relies on a hardware-level Compare-And-Swap instruction.

How to answer:

"For a simple shared counter, AtomicInteger is faster than synchronized because it avoids the overhead of acquiring/releasing a lock."

Q34

What is CAS (Compare-And-Swap)?

What it means:

CAS is a hardware-level atomic instruction: it compares a memory location's current value to an expected value, and only if they match does it swap in the new value — all in one uninterruptible step. If they don't match, the operation is retried.

How to answer:

"CAS lets you update a value atomically without a lock — check the old value hasn't changed, then swap; if it has changed, retry."

Q35

What is lock-free programming?

What it means:

An approach to thread-safety where threads never block waiting for a lock — instead they use techniques like CAS loops to keep retrying an operation until it succeeds, so no thread can hold up another indefinitely.

How to answer:

"It's thread-safety without blocking — usually built on CAS loops — which avoids issues like lock contention and deadlock, though the logic can get trickier to write."

Q36

Why are atomics faster than synchronized in many cases?

What it means:

Atomics use a single CPU-level CAS instruction with no thread blocking or context switching involved. synchronized, by contrast, may put a contending thread to sleep and wake it up later, which is much more expensive under contention.

How to answer:

"Atomics avoid the cost of thread blocking and context switches — for simple single-variable updates that's a lot cheaper than acquiring a monitor lock."

LEVEL 8 - Locks (ReentrantLock, ReadWriteLock, StampedLock)

Shows you know more than the basic synchronized keyword.

Q37

Difference between ReadLock and WriteLock?

What it means:

In a ReadWriteLock, multiple threads can hold the read lock at the same time since reading doesn't conflict with other reads. But only one thread can hold the write lock at a time, and while it's held, no thread — reader or writer — can access the resource.

How to answer:

"Reads are allowed to run concurrently with other reads, but a write needs exclusive access, blocking all readers and writers until it's done."

Q38

What is optimistic locking in StampedLock?

What it means:

StampedLock's optimistic read mode doesn't block writers at all — you read the data and get a "stamp", then afterward validate that stamp to check if a write happened during your read. If it did, you retry with a normal locked read.

How to answer:

"It's a hopeful, lock-free read — you assume no one wrote concurrently, then validate afterward, and only fall back to a real lock if that assumption turns out false."

Q39

Why use tryLock()?

What it means:

tryLock() attempts to acquire a lock without blocking forever — it returns immediately (or after a given timeout) with true or false. This is useful to avoid deadlock, since a thread can back off and retry instead of waiting indefinitely for a lock another thread might never release.

How to answer:

"I'd use tryLock() when I want to avoid getting stuck forever — if I can't get the lock in time, I back off, log it, or try an alternate path instead of risking deadlock."

LEVEL 9 - Volatile

Simple keyword, but interviewers use it to test if you truly understand visibility vs atomicity.

Q40

What does volatile do?

What it means:

It guarantees visibility: any write to a volatile variable by one thread is immediately visible to all other threads that read it, and it also prevents the compiler/CPU from reordering instructions around that variable.

How to answer:

"volatile guarantees every thread sees the latest written value immediately, and prevents instruction reordering around it — that's it, nothing more."

Q41

Does volatile make operations atomic?

What it means:

No. volatile only guarantees visibility and ordering, not atomicity. A compound operation like i++ on a volatile int is still not thread-safe, because it's actually a read-modify-write sequence that can be interleaved between threads.

How to answer:

This is the classic trap question — say it plainly: "No — volatile does NOT give you atomicity. i++ on a volatile variable can still lose updates under concurrent access."

Q42

Difference between volatile and synchronized?

What it means:

volatile only guarantees visibility/ordering for a single variable and involves no locking or blocking. synchronized provides mutual exclusion (only one thread executes the block at a time) plus visibility, for a whole block of code, not just one variable.

How to answer:

"volatile is lightweight and covers one variable's visibility; synchronized is heavier but gives full mutual exclusion over a block of code."

Q43

When should you use volatile?

What it means:

Use it for simple flags written by one thread and read by many — like a boolean "running" or "shutdown" flag — where you just need visibility and don't need any compound atomic operation.

How to answer:

"My go-to example is a shutdown flag: one thread sets it to true, worker threads just read it in a loop to know when to stop — no compound operation needed, so volatile is enough."

LEVEL 10 - Java Memory Model (JMM)

The hardest topic — but showing even a basic grasp impresses interviewers a lot.

Q44

What is the Java Memory Model?

What it means:

It's the specification that defines how threads interact through memory — specifically, what guarantees you get (or don't get) about when a write made by one thread becomes visible to another thread, and in what order operations can appear to happen.

How to answer:

"JMM is the rulebook that defines visibility and ordering guarantees between threads sharing memory — without it, the compiler/CPU are free to optimize in ways that break naive multithreaded assumptions."

Q45

What is visibility?

What it means:

Visibility is about whether a write made by one thread is guaranteed to be seen by another thread. Each CPU core may cache variables locally, so without proper synchronization a thread can keep reading a stale, cached value forever.

How to answer:

"Without synchronization or volatile, one thread's update might just sit in another core's cache and never become visible to a different thread — that's a visibility problem."

Q46

What is reordering?

What it means:

The JVM and CPU are allowed to execute instructions in a different order than they appear in the source code, as long as the result looks correct from a single thread's own perspective. This can cause unexpected behavior when multiple threads observe the reordered effects.

How to answer:

"Reordering is a performance optimization that's invisible within one thread but can surface as weird bugs across threads — that's exactly what happens-before rules and volatile are there to prevent."

Q47

What is the Happens-Before relationship?

What it means:

It's a set of rules the JMM defines that guarantee: if action A "happens-before" action B, then everything A did (including memory writes) is visible to B. For example, unlocking a monitor happens-before a later thread locking that same monitor.

How to answer:

Give a concrete example: "A thread's write before releasing a lock happens-before another thread's read after acquiring that same lock — that's the guarantee that makes synchronized safe for visibility, not just mutual exclusion."

LEVEL 11 - Deadlock

Always comes with a live example request — be ready to describe or draw one.

Q48

What is deadlock?

What it means:

A situation where two or more threads are each waiting for a lock that the other thread is holding, so none of them can ever proceed — they're stuck waiting forever.

How to answer:

"Classic example: Thread A holds Lock1 and wants Lock2; Thread B holds Lock2 and wants Lock1. Neither can move — that's deadlock."

Q49

How do you prevent it?

What it means:

The most common technique is to always acquire multiple locks in the same fixed, agreed-upon order across every thread. Other approaches: use tryLock() with a timeout, hold locks for as short a time as possible, or minimize how many locks you need at once.

How to answer:

"I enforce a consistent lock ordering everywhere in the code, and where that's not practical I use tryLock() with a timeout so a thread can back off instead of waiting forever."

Q50

Difference between deadlock, livelock, and starvation?

What it means:

Deadlock: threads are stuck waiting on each other forever, making zero progress. Livelock: threads keep actively changing state in response to each other, but still never make real progress (like two people stepping side to side to avoid each other in a hallway). Starvation: one thread never gets the CPU or a lock because other threads keep getting priority over it.

How to answer:

Use the hallway analogy for livelock — interviewers like it: "Livelock is like two people repeatedly stepping the same way trying to avoid each other in a corridor — busy, but stuck."

Q51

How do you detect deadlocks?

What it means:

Take a thread dump (e.g. using jstack) — the JVM itself can detect cyclic lock-wait dependencies and will report them directly in the dump. Tools like VisualVM or JConsole can visualize this too.

How to answer:

"In production I'd take a thread dump with jstack — the JVM actually identifies and reports deadlock cycles for you directly in the dump output."

LEVEL 12 - ForkJoin Framework

Less common than Executor/CompletableFuture, but shows deeper knowledge if asked.

Q52

What is Work Stealing?

What it means:

In a ForkJoinPool, each worker thread has its own task deque. When a thread finishes its own tasks, instead of sitting idle, it "steals" tasks from the back of another busy thread's deque — keeping all CPU cores productively busy.

How to answer:

"Idle worker threads steal queued subtasks from busier threads instead of sitting idle — that's what keeps ForkJoinPool efficient for divide-and-conquer workloads."

Q53

Difference between ThreadPoolExecutor and ForkJoinPool?

What it means:

ThreadPoolExecutor is built for independent tasks pulled from one shared queue. ForkJoinPool is built for divide-and-conquer style tasks that recursively split into smaller subtasks — each worker has its own deque, and idle workers steal work from others.

How to answer:

"ThreadPoolExecutor is for a flat pile of independent jobs; ForkJoinPool is designed specifically for tasks that split themselves into smaller subtasks recursively."

Q54

Where is ForkJoin used?

What it means:

It's used for CPU-intensive problems that can be recursively broken into smaller independent pieces — for example, parallel sorting/searching algorithms, and Java 8's parallel streams use a shared ForkJoinPool internally under the hood.

How to answer:

"A concrete example interviewers like: Java's parallelStream() actually runs on the common ForkJoinPool behind the scenes."

LEVEL 13 - Virtual Threads (Java 21)

Very trendy right now — expect at least one question on this in any recent interview.

Q55

What are Virtual Threads?

What it means:

Lightweight threads introduced in Java 21 that are managed by the JVM itself rather than mapped directly to OS threads. Because they're so cheap, you can create millions of them, unlike traditional platform threads which are expensive and limited in number.

How to answer:

"Virtual threads are JVM-managed, extremely lightweight threads — I can spin up huge numbers of them for I/O-heavy work without exhausting system resources the way I would with platform threads."

Q56

Platform Thread vs Virtual Thread?

What it means:

A platform thread maps 1:1 to an actual OS thread — expensive to create, limited in how many you can have running. A virtual thread is scheduled by the JVM on top of a small pool of "carrier" platform threads, so it's far cheaper and you can have vastly more of them.

How to answer:

"Platform threads are OS-backed and expensive; virtual threads are JVM-scheduled on top of a few carrier threads, so they're nearly free to create."

Q57

Are Virtual Threads faster?

What it means:

Not for CPU-bound computation — they don't give you more CPU cores. Their real benefit is massively improved throughput for I/O-bound or blocking workloads, because blocking a virtual thread doesn't block the underlying OS thread — the carrier thread is freed up to run other virtual threads.

How to answer:

"Not faster for pure computation — the win is throughput for blocking I/O workloads, since blocking a virtual thread frees the carrier thread to serve someone else."

Q58

Can synchronized block Virtual Threads?

What it means:

Historically yes — using synchronized inside a virtual thread used to "pin" it to its carrier thread while blocked, preventing the carrier from being reused by other virtual threads and hurting scalability. This has been largely improved in newer JDK versions, but it's still worth knowing as a caveat.

How to answer:

"It's a known caveat — older JDKs would pin the carrier thread when synchronized blocked inside a virtual thread. It's worth mentioning you're aware of it even though newer JDKs have improved this."

Q59

When should you avoid Virtual Threads?

What it means:

Avoid them for CPU-bound heavy computation, since they offer no advantage over platform threads there. Also be cautious with legacy code that relies heavily on ThreadLocal or native code that pins threads, since that can undercut the benefits.

How to answer:

"I wouldn't reach for virtual threads for a CPU-bound number-crunching task, or in code that's heavily reliant on ThreadLocal / native pinning — platform threads with a fixed pool make more sense there."

LEVEL 14 - Structured Concurrency

Newer topic, but increasingly asked at senior/backend interviews.

Q60

Why was Structured Concurrency introduced?

What it means:

To treat a group of related concurrent subtasks as one single unit of work with a clear lifetime — instead of manually managing separate futures/threads whose success, failure, and cancellation you have to wire together yourself.

How to answer:

"It brings the same discipline we already have for sequential code — a task's children don't outlive the task itself — into concurrent code, so nothing gets orphaned."

Q61

What problems does it solve?

What it means:

It fixes issues common with raw ExecutorService/CompletableFuture usage: threads that leak/keep running after their parent task is done, unclear task lifetimes, and messy, easy-to-miss error propagation across multiple concurrent subtasks.

How to answer:

"The main pain points it solves are orphaned threads and tangled error handling when you fire off several related async subtasks manually."

Q62

How is cancellation handled?

What it means:

If one subtask fails, or the enclosing scope is closed/cancelled, structured concurrency automatically cancels all the sibling subtasks and propagates the failure cleanly to the parent, instead of leaving other subtasks running in the background.

How to answer:

"If one subtask fails, the scope automatically cancels its siblings and the failure bubbles up to the parent — you don't have to manually track and cancel each one."

LEVEL 15 - ThreadLocal & Scoped Values

Practical topic — great for showing real production experience.

Q63

What is ThreadLocal?

What it means:

ThreadLocal gives each thread its own independent, isolated copy of a variable. Multiple threads can each read/write "the same" ThreadLocal variable without interfering with each other, because under the hood each thread actually has its own separate value.

How to answer:

"A common real use is storing a per-request user/context object in a web app — each thread handling a request gets its own isolated copy without needing synchronization."

Q64

What are memory leaks with ThreadLocal?

What it means:

In environments with reused threads (like a thread pool), if you don't call remove() after you're done, the value stays attached to that thread forever, since the thread itself never dies and gets reused for future tasks — this silently leaks memory over time.

How to answer:

"The fix I always apply is calling threadLocal.remove() in a finally block after use — especially critical in pooled-thread environments like web servers, where threads are long-lived and reused."

Q65

Why are Scoped Values preferred with Virtual Threads?

What it means:

Scoped Values are immutable and automatically cleaned up the moment their defined scope ends, avoiding both the manual remove() requirement and the inheritance overhead that ThreadLocal carries — which matters a lot when you might be spinning up millions of cheap virtual threads.

How to answer:

"Scoped Values were designed with virtual threads in mind — they auto-clean and avoid ThreadLocal's per-thread memory overhead, which becomes a real concern when you have millions of threads instead of a small pool."

QUICK COMPARISON ROUND

A few extra 'X vs Y' pairs interviewers love as rapid-fire warm-up questions — quick one-liners to have ready.

Q66

Thread vs Runnable?

What it means:

Thread is an actual class representing a real thread of execution. Runnable is just a functional interface describing the task/work to be done — you hand a Runnable to a Thread (or an ExecutorService) to actually run it.

How to answer:

"Runnable is the 'what to do', Thread is the 'who does it' — I almost always write Runnables and let a Thread or ExecutorService run them."

Q67

HashMap vs ConcurrentHashMap — the one-line version?

What it means:

HashMap has no synchronization and isn't safe for concurrent use; ConcurrentHashMap is thread-safe via fine-grained locking on parts of the map, and additionally doesn't allow null keys or values.

How to answer:

"Same data structure idea, but ConcurrentHashMap adds thread-safety through fine-grained locking, plus it disallows nulls — HashMap doesn't."

Q68

AtomicInteger vs synchronized — the one-line version?

What it means:

AtomicInteger uses lock-free CAS to safely update a single variable, which is fast under contention. synchronized locks an entire block of code using a monitor, which is more general-purpose but has more overhead for something as simple as a counter.

How to answer:

"For a plain counter, AtomicInteger wins on performance because it's lock-free; synchronized is the right call when I need to protect more than one variable or a multi-step operation atomically."

FREQUENTLY ASKED CODING QUESTIONS

These come up as live-coding or whiteboard exercises. You're not expected to recite full working code from memory in every interview — you need to clearly explain the approach and be able to sketch the key logic.

C1

Print Even and Odd Numbers using two threads

Approach:

Use one shared lock object and a boolean flag (or the current number's parity) that both threads check. Each thread checks if it's its turn; if not, it calls wait(). After printing, it flips the flag and calls notify() so the other thread wakes up.

How to explain it to the interviewer:

"I'd use a shared monitor with wait()/notify() and a turn-tracking flag — each thread only prints when the flag says it's their turn, then hands off to the other."

C2

Producer-Consumer using BlockingQueue

Approach:

BlockingQueue already handles all the waiting logic for you: the producer calls put() which automatically blocks if the queue is full, and the consumer calls take() which automatically blocks if the queue is empty. No manual wait()/notify() needed.

How to explain it to the interviewer:

"This is the version I'd actually use in production — BlockingQueue does the blocking/waiting internally, so my code stays simple and less error-prone than hand-rolled wait/notify."

C3

Producer-Consumer using wait()/notify()

Approach:

Use a shared bounded buffer inside a synchronized class. The producer's add method waits (in a while loop) while the buffer is full, adds an item, then calls notifyAll(). The consumer's remove method waits while the buffer is empty, removes an item, then calls notifyAll().

How to explain it to the interviewer:

"This is the classic manual version — I always check the wait condition in a while loop, not an if, to guard against spurious wakeups and stale conditions."

C4

Print A B C repeatedly using three threads

Approach:

Similar turn-passing idea, extended to three participants: either a shared 'turn' variable (0,1,2) checked with wait/notifyAll in a loop, or cleaner — give each thread its own Semaphore, and after printing, each thread releases the permit for the next thread in sequence.

How to explain it to the interviewer:

"I'd lean toward the Semaphore version for 3+ threads — it reads more clearly than juggling a shared turn variable with wait/notifyAll."

C5

Print 1-100 using two threads alternately

Approach:

Same turn-passing pattern as even/odd — a shared lock and a boolean 'isOddTurn' flag; each thread only proceeds and prints when it's its turn, then flips the flag and notifies the other thread.

How to explain it to the interviewer:

"Functionally the same pattern as the even/odd problem — I'd point that out to the interviewer to show I recognize the underlying pattern rather than treating it as a brand-new problem."

C6

Implement your own thread pool (basic version)

Approach:

Create a BlockingQueue for pending tasks, and a fixed number of worker threads that each loop forever calling queue.take() and running whatever Runnable they get. Submitting a task just means adding it to the queue.

How to explain it to the interviewer:

"I'd explain that this is essentially a simplified reimplementation of what ThreadPoolExecutor already does internally — a task queue plus a fixed set of worker threads pulling from it."

C7

Implement a Singleton that is thread-safe

Approach:

Best practical approaches: the Initialization-on-Demand Holder idiom (a static nested class, lazily loaded by the JVM's own thread-safe classloading) or an enum-based singleton (inherently thread-safe and serialization-safe). Double-checked locking with a volatile field also works but is more error-prone to write correctly.

How to explain it to the interviewer:

"My default answer is the static holder idiom or enum singleton — both are thread-safe without needing any explicit locking, since they rely on the JVM's guaranteed thread-safe class initialization."

C8

Demonstrate a deadlock and then fix it

Approach:

Demonstrate: Thread A locks Lock1 then tries to lock Lock2; Thread B locks Lock2 then tries to lock Lock1 — run them concurrently and they'll hang forever. Fix: make both threads acquire locks in the same order (e.g., always Lock1 before Lock2).

How to explain it to the interviewer:

"I'd draw the two-thread, two-lock example on the whiteboard, then explain the fix is simply enforcing a consistent global lock ordering across every thread that needs both locks."

C9

Use CompletableFuture to call three services in parallel

Approach:

Kick off three independent calls with CompletableFuture.supplyAsync() (each hitting a different service), then use CompletableFuture.allOf(f1, f2, f3) to wait for all three to finish, and finally read each individual result with .join() or .get().

How to explain it to the interviewer:

"I'd fire all three supplyAsync() calls up front so they truly run in parallel, then combine with allOf() rather than calling .get() on each one sequentially, which would accidentally make them run one after another."

C10

Build a simple rate limiter using concurrency primitives

Approach:

A quick approach: use a Semaphore initialized with N permits representing your rate limit; each request must acquire a permit before proceeding and release it after. For a proper time-window limiter, combine an AtomicInteger token count with a ScheduledExecutorService that resets/refills the token count on a fixed interval (token bucket pattern).

How to explain it to the interviewer:

"For a simple version I'd use a Semaphore with a fixed number of permits; if they want a real time-based limiter, I'd explain the token bucket pattern using AtomicInteger plus a ScheduledExecutorService to refill tokens periodically."

Final Prep Checklist

- Can you explain the difference between wait() and sleep() without hesitating? (Almost always asked)
- Can you explain how ConcurrentHashMap avoids locking the whole map?
- Can you explain CAS in your own words with a tiny example?
- Can you draw a 2-thread, 2-lock deadlock and explain the fix?
- Can you explain the difference between thenApply() and thenCompose()?
- Can you explain visibility vs atomicity using the volatile example?
- Can you explain why Virtual Threads help I/O-bound workloads specifically, not CPU-bound ones?
- Can you talk through the Producer-Consumer problem both with BlockingQueue AND with raw wait()/notify()?

Good luck! Go through this guide question-by-question, answer out loud each time before reading the script — that's what makes it stick for the actual interview.